Development Activity 4 — Dependency Mapping & Failure Analysis

FAST-NUCES, Lahore
Secure Software Development (SSD)
Fall 2026 Semester Project

Project Submission Details

Project Name:	**SafeHire: Verified Credential & Skill-Match Recruitment Portal**
Group Members:	1. **Muhammad Umar Afzaal** (Roll No: **23F-3106**) | 2. **Musa Rehan** (Roll No: **23F-3093**)
Live / Local URL:	**https://safehire-secdev.vercel.app/** | Local: **http://localhost:3000/** (Vite)
GitHub Repo:	**https://github.com/muhammadumarafzaal/safehire-secdev**

Part 1 — Identify Your System Components

Symbol	Component Name	Main Responsibility
C1	**Authentication & Role-Based Access Control Service**	Manages user login/logout, role selection (Student vs. Placement Officer), session state, credential validation via demo user store, privacy consent, and protected route guardrails preventing unauthenticated dashboard access.
C2	**Opportunity Data Store & Validation Engine**	Centralized data layer holding verified internship listings (opportunitiesData.js), demo user credentials (demoUsers.js), domain categories, work modes, and all shared validation/sanitization utilities (sanitizeInput, validateOpportunityForm, validateSearchQuery, generateOpportunityDigest).
C3	**Post Verified Opportunity Module (Module 1)**	Enterprise requisition interface enabling authorized users to compose, validate, cryptographically sign (SHA-256 HMAC digest), and publish new verified internship opportunities into shared reactive state.
C4	**Browse & Search Opportunities Module (Module 2)**	Full catalog explorer with real-time keyword search (query sanitization & length guardrails), multi-facet filtering, interactive cryptographic verification proof inspection modal, and bookmark state machine.
C5	**Application Orchestrator & Shared State Manager**	Central orchestrator (App.jsx) managing global reactive state (opportunities array, user session, toast notifications, audit drawer), page routing, cross-module data flow, and component composition/rendering.
C6	**Navigation, Layout & Feedback UI Layer**	Encompasses Navbar (role-aware links & logout), Footer (cross-module navigation), Toast notification stack, Audit Ledger Drawer, and Design Token System (tokens.css) enforcing visual consistency across all pages.

Part 2 — Identify Direct Dependencies

Component	Directly Depends On	Reason
C1 (Auth & RBAC)	**C2** (Data Store), **C5** (Orchestrator)	Login page imports credentials from DEMO_USERS in C2; receives setUser and setCurrentPage callbacks from C5.
C2 (Data Store)	*(None)*	Self-contained data and utility module with no runtime dependencies on other components; exports static data arrays and pure validation functions.
C3 (Post Module)	**C2** (Data Store), **C5** (Orchestrator)	Invokes validateOpportunityForm(), sanitizeInput(), and generateOpportunityDigest() from C2; calls onAddOpportunity from C5.
C4 (Browse Module)	**C2** (Data Store), **C5** (Orchestrator)	Uses validateSearchQuery() from C2; reads CATEGORIES and WORK_MODES; receives live opportunities array as prop from C5.
C5 (Orchestrator)	**C2** (Data Store), **C6** (UI Layer)	Imports INITIAL_OPPORTUNITIES and DEMO_USERS from C2 for state initialization; composes C6 sub-components (Navbar, Footer, AuditDrawer, Toast) into the app shell.
C6 (UI Layer)	**C5** (Orchestrator)	C6 components (Navbar, Toast, AuditDrawer) receive their state and callbacks (currentPage, user, setCurrentPage, addToast) as props from C5.

Development Activity 4 — Dependency Mapping & Failure Analysis

Part 3 — Construct the Dependency Matrix

Reading: Row component **depends on** Column component. A **1** in row Cx, column Cy means 'Cx depends on Cy.' Diagonal is always **0**.

Row / Col	C1	C2	C3	C4	C5	C6
C1	0	1	0	0	1	0
C2	0	0	0	0	0	0
C3	0	1	0	0	1	0
C4	0	1	0	0	1	0
C5	0	1	0	0	0	1
C6	0	0	0	0	1	0

Matrix Analysis: Column sums (inbound dependencies): C1: 0 | **C2: 4** | C3: 0 | C4: 0 | **C5: 4** | C6: 1. C2 is the only component with **zero outbound dependencies** (foundational leaf node).

Part 4 — Analyze Component Failures

Failure Scenario 1: C2 — Opportunity Data Store & Validation Engine

Direct Dependents:	**C1** (Authentication), **C3** (Post Module), **C4** (Browse Module), **C5** (App Orchestrator) — 4 of 5 components.
Indirect Cascade:	C5 depends on C2 and C6 depends on C5, so C2's failure cascades through C5 to C6. **Every component** in the system is transitively affected.
Functions Disrupted:	• **Total login failure** — no credential definitions exist. • **No opportunity listings** — initial dataset undefined; catalog renders empty. • **No form validation** — validateOpportunityForm() and sanitizeInput() unavailable; security guardrails removed. • **No search sanitization** — validateSearchQuery() unavailable; injection protection lost. • **No cryptographic digests** — generateOpportunityDigest() cannot produce tamper-evident signatures. • **Complete application crash** — React render tree throws runtime error at mount, resulting in blank white page.

Failure Scenario 2: C5 — Application Orchestrator & Shared State Manager

Direct Dependents:	**C1** (Authentication), **C3** (Post Module), **C4** (Browse Module), **C6** (Navigation & UI Layer) — 4 of 5 components.
Indirect Cascade:	C5 sits at the top of the component tree. Although C2 does not depend on C5, C2 becomes effectively useless in isolation because no component can render its data without the App Orchestrator wiring it through props. **Total system unavailability** results.
Functions Disrupted:	• **Complete render failure** — no pages are mounted; browser shows blank screen. • **Loss of shared state** — reactive opportunities[], user session, and toast queue cease to exist. • **No page routing** — navigation between Home, Login, Dashboard, Module 1, and Module 2 is impossible. • **No cross-module data flow** — Module 1 cannot dispatch postings to Module 2. • **No authentication lifecycle** — login, logout, and protected route enforcement all fail.

Development Activity 4 — Dependency Mapping & Failure Analysis

FAST-NUCES, Lahore
Secure Software Development (SSD)

Part 5 — Identify a Critical Component

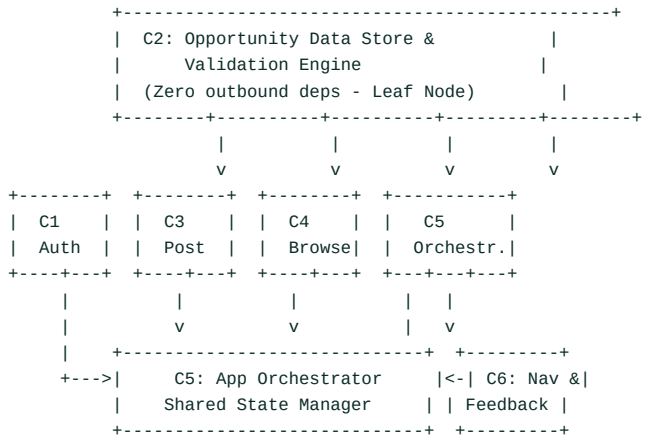
Selected Component:	**C2 — Opportunity Data Store & Validation Engine**
Direct Dependents:	C1 (Authentication), C3 (Post Module), C4 (Browse Module), and C5 (App Orchestrator) — **4 out of 5** other components.
Why Critical:	• **Maximum cascade depth:** C5 depends on C2, and C6 depends on C5, so C2's failure propagates transitively to collapse the *entire system*. • **Security-critical functions:** C2 houses all input sanitization and anti-XSS validation functions. Its absence does not merely break features — it **removes security guardrails**, potentially allowing malicious payloads to propagate. • **Single source of truth:** C2 is the only component with zero outbound dependencies and provides all business data (opportunities, credentials, categories). Unlike C5, C2's data and validation logic are irreplaceable without rewriting the application's business rules.

Part 6 — Propose a Failure-Handling Strategy

Component:	**C2 — Opportunity Data Store & Validation Engine**
Proposed Strategy:	• **Embedded Fallback Defaults:** Each consuming component (C1, C3, C4, C5) defines local hardcoded fallback constants (empty arrays for opportunities, guest-mode user objects, inline basic validation rules) that activate when C2 import fails or returns undefined. • **Safe Error Boundary Wrapper:** A top-level React ErrorBoundary catches runtime exceptions (failed imports, null references) and renders an informative error screen instead of a blank white page. • **Inline Validation Fallback (Fail-Closed):** If validateOpportunityForm() or sanitizeInput() are unavailable, the Post Opportunity form **disables the submit button entirely** with a warning: 'Submission temporarily disabled — security validation engine offline.' This ensures the system fails closed (denying potentially unsafe input) rather than failing open. • **Monitoring & Alerting:** Log the data layer failure to the browser console with structured error objects for rapid developer diagnosis.
Impact Reduction:	The fallback defaults and error boundary prevent the entire application from crashing, allowing informational content (Home Page, Navigation) to remain accessible. The fail-closed validation strategy ensures the system never silently accepts malicious input when validators are offline. Monitoring accelerates diagnosis and reduces mean time to recovery (MTTR).
Partial Continuity:	The entire system would **not** need to stop. Home Page and Navigation UI continue rendering static content. Login Page shows graceful degradation message. Post Opportunity and Browse Catalog enter read-only degraded mode. Only data-dependent transactional operations are paused until C2 is restored.

Part 7 — Dependency Graph

Arrow from A to B means '**A depends on B**.' Graph matches the dependency matrix above.



Dependency Paths: C1->C2,C5 | C2->(none) | C3->C2,C5 | C4->C2,C5 | C5->C2,C6 | C6->C5

Final Reflection

Dependency mapping has fundamentally deepened our understanding of SafeHire's reliability and security posture by revealing that our **Opportunity Data Store & Validation Engine (C2)** is the single most critical foundation — four of the five remaining components depend on it directly, and its failure cascades transitively to collapse the entire system. This exercise also exposed a critical security insight: C2 houses all input sanitization and anti-XSS validation functions, meaning its unavailability does not merely break features but **removes security guardrails**, potentially opening the system to injection attacks if any component attempts to operate in a degraded mode without validation. Armed with this analysis, we now prioritize C2 as the component requiring the most rigorous defensive design (error boundaries, fallback defaults, fail-closed submission policies) and have identified the bidirectional dependency cycle between C5 and C6 as a structural coupling that should be refactored in future iterations to improve fault isolation.

Submission Requirements Checklist

Deliverable	Status
Part 1: List of 6 system components and responsibilities	✓ **Complete**

Part 2: Direct dependency table with reasons	✓ **Complete**
Part 3: Completed 6x6 dependency matrix with analysis	✓ **Complete**
Part 4: Two component-failure scenario analyses (C2 and C5)	✓ **Complete**
Part 5: Critical component identification (C2) with justification	✓ **Complete**
Part 6: Failure-handling strategy with graceful degradation	✓ **Complete**
Part 7: Dependency graph (ASCII art representation)	✓ **Complete**
Final Reflection: Reliability & security insights	✓ **Complete**